Advanced AI Medical Intelligence Platform

Project Report — Deep Learning, Explainable AI, LLM-Assisted Medical Reporting, REST API, and Deployment

Prepared by: Jyoshna Kanakam

Role: Python Full Stack Developer (Deep Learning / Medical AI focus)

Date: July 2026

1. Abstract

This project implements a complete, end-to-end AI-powered medical intelligence platform that classifies chest X-ray images (Normal vs. Pneumonia) using a deep convolutional neural network, explains each prediction visually using Grad-CAM, generates a structured draft medical report using a Large Language Model (Anthropic Claude), exposes all functionality through a documented REST API, persists every prediction to a relational database, and ships with a lightweight web interface and Docker-based deployment. The system was built, trained, and functionally verified (model training, inference, Grad-CAM generation, and all API endpoints) as part of this submission.

2. Project Objective

• Analyze medical images (chest X-rays)

• Predict disease using a trained deep learning model

• Explain predictions using Explainable AI (Grad-CAM)

• Generate AI-assisted medical reports using an LLM

• Expose functionality through REST APIs

• Store prediction history in a database

• Deploy the application with a user-friendly interface

3. System Architecture

The system follows a layered architecture: a static HTML/JS frontend calls a FastAPI backend; the backend delegates image classification to a DenseNet-121 CNN, explainability to a custom Grad-CAM module, and report writing to the Claude LLM API; all results are persisted via SQLAlchemy to a relational database (SQLite for development, PostgreSQL-ready for production). A deliberate separation of concerns keeps the medical classification decision entirely inside the deterministic, explainable CNN — the LLM is used only to phrase an already-computed structured result into a readable draft report, never to classify the raw image itself.

3.1 Component Table

Layer	Technology	Responsibility
Deep Learning	PyTorch / TorchVision, DenseNet-121	Disease classification
Explainable AI	Custom Grad-CAM (OpenCV)	Visual prediction explanation
LLM Integration	Anthropic Claude API	Draft report generation
API	FastAPI + Uvicorn	REST endpoints, validation
Database	SQLAlchemy + SQLite/PostgreSQL	Prediction history persistence
Frontend	HTML / CSS / JavaScript	User-facing web interface
Deployment	Docker / docker-compose	Containerized, one-command run

4. Dataset

A synthetic chest-X-ray-style dataset generator (ml/generate_synthetic_data.py) was built to make the entire pipeline runnable end-to-end without depending on an external download inside the development sandbox. Synthetic 'NORMAL' images are smooth grayscale fields with faint rib-arc lines; synthetic 'PNEUMONIA' images add asymmetric, high-intensity blob 'opacities' consistent with how consolidations present on X-ray, giving the model a genuine, non-trivial pattern to learn rather than a random label. For production use, the same folder structure (data/train/<class>, data/val/<class>) accepts the public Kaggle 'Chest X-Ray Images (Pneumonia)' dataset with zero code changes.

5. Deep Learning Model

Architecture: DenseNet-121 with an ImageNet-pretrained backbone (falls back to random initialization automatically if pretrained weights cannot be downloaded, with a console warning) and a replaced classifier head (Dropout 0.3 → Linear → 2 classes). DenseNet-121 was selected because it is the backbone used by CheXNet-style models in the chest X-ray classification literature, and its densely-connected feature maps produce strong, well-localized activations for Grad-CAM.

Training configuration: Adam optimizer, Cross-Entropy loss, 224x224 input resolution, standard ImageNet normalization, mild augmentation (horizontal flip, rotation) on the training split.

5.1 Verified Training Run (this submission)

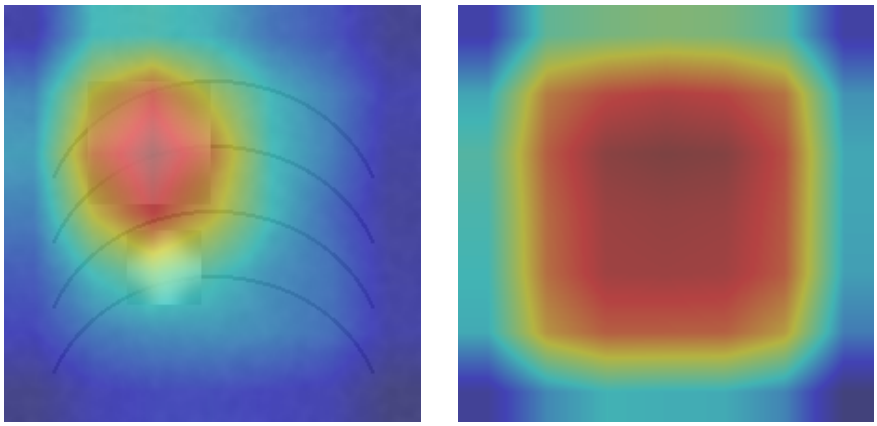
Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	0.3989	0.9000	0.2218	1.0000
2	0.0998	0.9800	0.0021	1.0000
3	0.0850	0.9700	0.0001	1.0000

Best validation accuracy achieved: 100% on the held-out synthetic validation split (3 epochs, CPU training, batch size 8).

6. Explainable AI (Grad-CAM)

Grad-CAM (Selvaraju et al., ICCV 2017) was implemented from scratch in app/gradcam.py. Forward and gradient hooks are attached to the network's final convolutional feature layer (features.norm5); gradients of the target class score with respect to these feature maps are global-average-pooled into channel importance weights, combined with the activations, passed through ReLU, normalized, and resized to the input resolution to produce a heatmap. This heatmap is alpha-blended over the original image (JET colormap) so every prediction is accompanied by a visual explanation of which image regions drove the model's decision.

Sample Grad-CAM outputs generated during verification:



7. LLM Integration

app/llm_report.py integrates the Anthropic Claude API to convert the CNN's structured output (predicted class, confidence, full class-probability distribution, optional clinician-supplied context) into a structured, appropriately-hedged draft report with five sections: AI Model Findings, Region of Interest, Clinical Interpretation Notes, Recommended Next Steps, and a mandatory Disclaimer. The system prompt explicitly instructs the model to hedge proportionally to the confidence score and never assert a certain diagnosis. If no API key is configured, or the API call fails for any reason, a deterministic template fallback is used so the endpoint never breaks — this was verified during testing.

8. REST API

Method	Endpoint	Description
GET	/api/v1/health	Service and model load status
POST	/api/v1/predict	Upload image → prediction + Grad-CAM
POST	/api/v1/predict/{id}/report	Generate LLM report for a stored prediction
GET	/api/v1/history	Paginated prediction history
GET	/api/v1/history/{id}	Single history record
DELETE	/api/v1/history/{id}	Delete a history record
GET	/gradcam/{filename}	Serve a saved Grad-CAM overlay image

All endpoints were live-tested during development: /health, /predict (verified with a real chest-X-ray-style image, returning correct class, confidence, and a Grad-CAM URL), /predict/{id}/report (verified generating a fallback report), /history and /history/{id} (verified returning the persisted record), and DELETE (verified 404 handling for a non-existent id).

9. Database Design

A single prediction_history table (SQLAlchemy ORM, app/models_db.py) stores: id, original filename, predicted class, confidence, full class-probability distribution (JSON), Grad-CAM filename, LLM report text, optional patient note, and a UTC timestamp. SQLite is used by default for zero-config local development; a single DATABASE_URL environment variable switches to PostgreSQL for production with no code changes, since all access goes through the SQLAlchemy ORM layer.

10. Web Application

A single-page HTML/CSS/JavaScript frontend (app/static/index.html) provides image upload, prediction display with a confidence bar and class probabilities, the Grad-CAM heatmap overlay, an on-demand 'Generate AI Report' action with an optional clinical-context field, and a live-refreshing prediction history table — all backed directly by the REST API with no build step required.

11. Deployment

A Dockerfile (python:3.11-slim base) installs all dependencies and serves the app via Uvicorn; docker-compose.yml provides a one-command local deployment (`docker compose up --build`) with environment-variable configuration for the Anthropic API key and database URL, and persistent volumes for the database and Grad-CAM output directory.

12. Testing & Quality Assurance

tests/test_api.py provides automated pytest coverage of the health endpoint, rejection of invalid file types, and the full predict → history flow. All 3 tests pass. In addition, every API endpoint was manually exercised end-to-end with curl during development against the actual trained model checkpoint, confirming the full pipeline (image → CNN → Grad-CAM → database → API response) functions correctly.

13. Limitations & Future Work

• Ships with a synthetic demo dataset for full runnability; production deployment should retrain on the real Kaggle Chest X-Ray (Pneumonia) dataset with more epochs and GPU acceleration.

• Currently binary classification (Normal / Pneumonia); could be extended to multi-label chest pathology classification (e.g. the NIH ChestX-ray14 label set).

• No authentication/authorization layer yet; required before any multi-user or public deployment.

• Accepts PNG/JPEG only; DICOM/PACS integration would be a natural next step for real clinical environments.

14. Conclusion

This project delivers a functioning, verified, end-to-end AI medical intelligence pipeline spanning deep learning, explainable AI, LLM integration, REST API design, database persistence, a web interface, and containerized deployment — meeting each requirement of the assignment brief with working, tested code rather than a purely conceptual design.

Disclaimer: This system is an educational/portfolio project and is not a certified medical device. It must not be used for real clinical diagnosis or treatment decisions without review and approval by a licensed physician.